高并发服务器（二）

epoll

epoll函数概念

epoll是Linux下多路复用IO接口select/poll的增强版本，提高程序在大量并发连接处理。epoll会复用文件描述符集合来传递结果而不用迫使开发者每次等待事件之前都必须重新准备要侦听的文件描述符集合，另外一点就是获取事件的时候，它无须遍历整个被侦听的描述符集，只要遍历那些被内核IO事件异步唤醒而加入ready队列的描述符集合就可以。

设置

1、查看一个进程打开最大数目的socket描述符（跟内存有关）：cat /proc/sys/fs/file_max

```
xingwenpeng@ubuntu:~/LinuxCode/epoll-13$ cat /proc/sys/fs/file-max
98092
```

2、设置最大打开文件描述符（开机重启有效）

sudo vi/etc/security/limits.conf

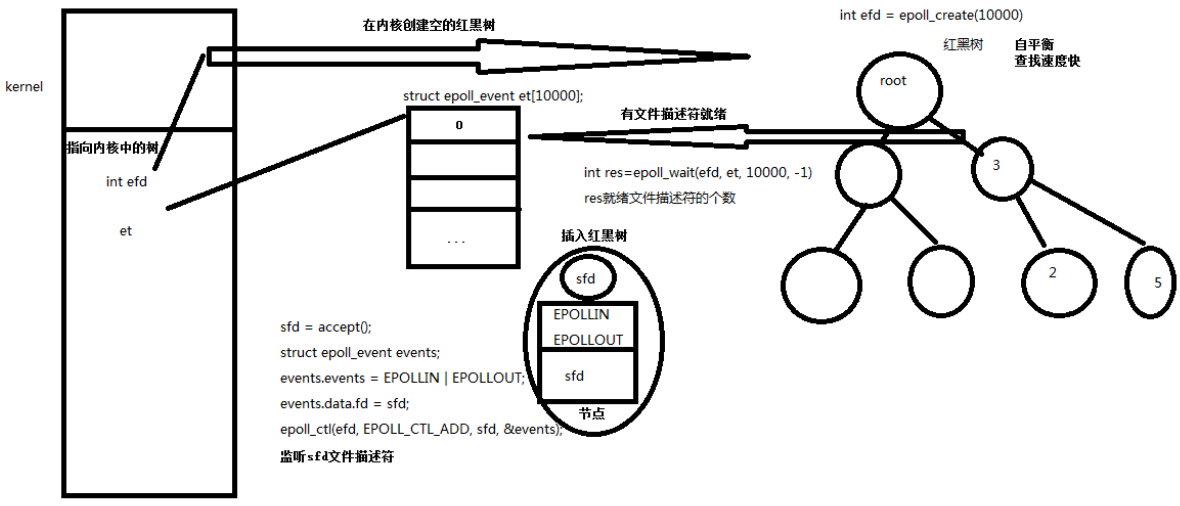
写入以下配置，soft软限制，hard硬限制(*代表所有用户)

*　　soft　　nofile　　65536

*　　hard　　nofile　　100000

Epoll API

epoll原理



1、epoll_create

创建一个epoll句柄，参数size用来告诉内核监听的文件描述符个数，跟内存大小有关。

```
int epoll_create(int size);


size:告诉内核监听的数目（现在size大小取决于内存的大小）
返回一个文件描述符，作为操作的句柄。
```

内部创建了红黑树，是一颗空树。

2、epoll_ctl

控制某个epoll监听的文件描述符上的事件：注册，修改，删除。

在红黑树中注册，修改，删除，监听的文件描述符

```
#include <sys/epoll.h>


int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event)


epfd：为epoll_creat的句柄


op：表示动作，用3个宏来表示：
    EPOLL_CTL_ADD(注册新的fd到epfd)，
    EPOLL_CTL_MOD(修改已经注册的fd的监听事件)，
    EPOLL_CTL_DEL(从epfd删除一个fd)；


event：告诉内核需要监听的事件
```

```
17
18
19 struct epoll_event
20 {
21     __uint32_t events; /* Epoll events */
22     epoll_data_t data; /* User data variable */
23 };
24
25
26 typedef union epoll_data
27 {
28     void *ptr;
29     int fd;
30     uint32_t u32;
31     uint64_t u64;
32 } epoll_data_t;
33
34
35 EPOLLIN ： 表示对应的文件描述符可以读（包括对端SOCKET正常关闭）
36 EPOLLOUT: 表示对应的文件描述符可以写
37 EPOLLPRI: 表示对应的文件描述符有紧急的数据可读（这里应该表示有带外数据到来）
38 EPOLLERR: 表示对应的文件描述符发生错误
39 EPOLLHUP: 表示对应的文件描述符被挂断；
40 EPOLLET ： 将EPOLL设为边缘触发(Edge Triggered)模式，这是相对于水平触发(Level Triggered)来i
41 EPOLLONESHOT ： 只监听一次事件，当监听完这次事件之后，如果还需要继续监听这个socket的话，需要
```

3、epoll_wait

等待所监控文件描述符上有事件的产生，类似于select()调用。

```
1 #include <sys/epoll.h>
2
3
4 int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);
5
6
7 events: 用来从内核得到事件的集合（数组，传出参数）（不在内核中，在用户内存上）；
8
9
```

```
maxevents：告之内核这个events有多大，这个maxevents的值不能大于创建epoll_create()时的size；


timeout：是超时时间
    -1：阻塞
     0：立即返回，非阻塞
    >0：指定微秒


返回值：成功返回有多少文件描述符就绪，时间到时返回0，出错返回-1
```

4、服务器端

4.1、server.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/epoll.h>
#include <errno.h>
#include "wrap.h"


#define MAXLINE 4096
#define SERV_PORT 8090
#define OPEN_MAX 65525




int main(int argc,char *argv[])
{
    int i,j,maxi,listenfd,connfd,sockfd;
    int n;
    ssize_t nready,efd,res;
    char buf[MAXLINE],str[INET_ADDRSTRLEN];
    socklen_t clilen;
    struct sockaddr_in cliaddr,servaddr;
```

```
27
28
29      struct epoll_event tep,ep[OPEN_MAX];
30
31
32      listenfd = Socket(AF_INET,SOCK_STREAM,0);
33
34
35      int opt = 1;
36      //设置socket属性 取消2msl保护
37      setsockopt(listenfd,SOL_SOCKET,SO_REUSEADDR,&opt,sizeof(opt));
38
39
40      bzero(&servaddr,sizeof(servaddr));
41      servaddr.sin_family = AF_INET;
42      servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
43      servaddr.sin_port = htons(SERV_PORT);
44
45
46      Bind(listenfd,(struct sockaddr *)&servaddr,sizeof(servaddr));
47
48
49      Listen(listenfd,128);
50
51
52      efd = epoll_create(OPEN_MAX);
53      if(efd == -1)
54          perr_exit("epoll_create");
55      tep.events = EPOLLIN;//读数据
56      tep.data.fd = listenfd;
57      res = epoll_ctl(efd,EPOLL_CTL_ADD,listenfd,&tep);
58      if(res == -1)
59          perr_exit("epoll_ctl");
60
61
62      for(;;)
63      {
64          nready = epoll_wait(efd,ep,OPEN_MAX,-1);
65          //阻塞监听
66          if(nready == -1)
67              perr_exit("epoll_wait");
68          for(i = 0; i < nready; ++i)
69          {
70              if(!(ep[i].events & EPOLLIN))//不是读事件跳过
```

```
                continue;
            if(ep[i].data.fd == listenfd)
            {
                clilen = sizeof(cliaddr);
                connfd = Accept(listenfd,(struct sockaddr *)&cliaddr,&clilen);
                printf("received from %s at PORT\n",inet_ntop(AF_INET,&cliaddr.sin_a
                tep.events = EPOLLIN;
                tep.data.fd = connfd;
                res = epoll_ctl(efd,EPOLL_CTL_ADD,connfd,&tep);
                if(res == -1)
                    perr_exit("epoll_ctl");
            }
            else
            {
                sockfd = ep[i].data.fd;
                n = Read(sockfd,buf,MAXLINE);
                if(n <= 0)
                {
                    res = epoll_ctl(efd,EPOLL_CTL_DEL,sockfd,NULL);
                    if(res == -1)
                        perr_exit("epoll_ctl");
                    Close(sockfd);
                    printf("client[%d] closed connection\n",j);
                }
                else
                {
                    for(j = 0; j < n; ++j)
                        buf[j] = toupper(buf[j]);
                    Writen(sockfd,buf,n);
                }
            }
        }
    }
    Close(listenfd);
    Close(efd);
    return 0;
}
```

5、客户端

5.1、client.c

```c
#include <stdio.h>
#include <unistd.h>
#include <strings.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include "wrap.h"


#define SERV_PROT 8090
#define SERV_IP "192.168.152.128"


int main(void)
{
    int cfd,len;
    struct sockaddr_in saddr;
    char buf[4096] = "helloworld";


    cfd = Socket(AF_INET,SOCK_STREAM,0);


    bzero(&saddr,sizeof(saddr));
    saddr.sin_family = AF_INET;
    inet_pton(AF_INET,SERV_IP,(void*)&saddr.sin_addr.s_addr);//目标器的ip地址
    saddr.sin_port = htons(SERV_PROT);


    Connect(cfd,(struct sockaddr *)&saddr,sizeof(saddr));


    while(fgets(buf,sizeof(buf),stdin) != NULL)
    {
        Write(cfd,buf,strlen(buf));
        len = read(cfd,buf,sizeof(buf));
        Write(STDOUT_FILENO,buf,len);
    }

```

```
41
42      Close(cfd);
43
44
45      return 0;
46  }
```

5.2、client_while.c

```
1  /* client.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <unistd.h>
6  #include <netinet/in.h>
7  #include "wrap.h"
8  #define MAXLINE 4096
9  #define SERV_PORT 8090
10
11
12 int main(int argc, char *argv[])
13 {
14      struct sockaddr_in servaddr;
15      char buf[MAXLINE] = "heel";
16      int sockfd, n;
17      if(argc < 2)
18      {
19          printf("./client IP\n");
20          exit(1);
21      }
22
23
24      while(1)
25      {
26          sockfd = Socket(AF_INET, SOCK_STREAM, 0);
27
28
29          bzero(&servaddr, sizeof(servaddr));
30          servaddr.sin_family = AF_INET;
```

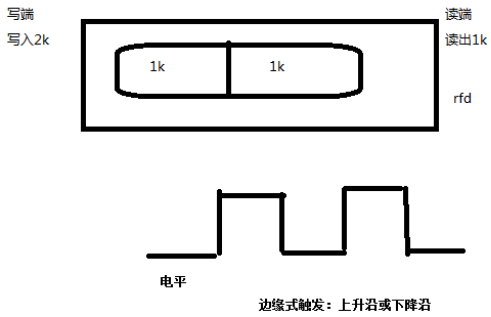
```
        inet_pton(AF_INET, argv[1], &servaddr.sin_addr);
        servaddr.sin_port = htons(SERV_PORT);


        Connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
        //Close(sockfd);
    }
    return 0;
}
```

再议epoll

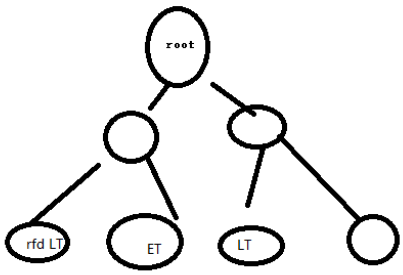
1、Epol提供了两种IO事件：

1、 电平触发（LT）默认是电平触发处理（只要有数据都会触发），当文件描述符没有处理完，在wait再次检测接着处理（多次通知）。

2、 边沿触发（ET）边缘触发只有数据到来，才触发，不管缓存区中是否还有数据（一次通知）。高并发，多链接



边缘式触发：上升沿或下降沿



具体到那个节点的是什么触发。

epoll_wait()
1.水平式触发　水平触发，第一次没有读完，下次返回继续读　多次通知
2.边缘式触发　一次通知

2、基于管道epoll ET触发模式

```
/*epoll_pipe.c*/
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <sys/epoll.h>
#include <errno.h>
#include <unistd.h>
#define MAXLINE 10
int main(int argc, char *argv[])
{
    int efd, i;
    int pfd[2];
    pid_t pid;
    char buf[MAXLINE], ch = 'a';
    pipe(pfd);
    pid = fork();
    if (pid == 0) {
        close(pfd[0]);//关闭读
        while (1) {
            //aaaa\n
            for (i = 0; i < MAXLINE/2; i++)
                buf[i] = ch;
            buf[i-1] = '\n';
            ch++;
            //bbbb\n
            for (; i < MAXLINE; i++)
                buf[i] = ch;
            buf[i-1] = '\n';
            ch++;
            write(pfd[1], buf, sizeof(buf));
            sleep(2);
        }
        close(pfd[1]);
    }
    else if (pid > 0) {
        struct epoll_event event;
        struct epoll_event resevent[10];
        int res, len, i = 0;


        close(pfd[1]);//关闭写
        efd = epoll_create(10);
        /* ET 边沿触发，默认是水平触发*/
        event.events = EPOLLIN | EPOLLET;
        //event.events = EPOLLIN;
        event.data.fd = pfd[0];
        epoll_ctl(efd, EPOLL_CTL_ADD, pfd[0], &event);
```

```
        while (1) {
            res = epoll_wait(efd, resevent, 10, -1);
            printf("%d\t", ++i);
            fflush(stdout);
            if (resevent[0].data.fd == pfd[0]) {
                len = read(pfd[0], buf, MAXLINE/2);
                write(STDOUT_FILENO, buf, len);
            }
        }
        close(pfd[0]);
        close(efd);
    }
    else {
        perror("fork");
        exit(-1);
    }
    return 0;
}
```

3、基于网络CS模型的epoll ET触发模式

3.1、server.c

```
#include <stdio.h>
#include <string.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <sys/epoll.h>
#include <unistd.h>


#define MAXLINE 10
#define SERV_PORT 8090


int main(void)
{
    struct sockaddr_in servaddr, cliaddr;
```

```
    struct sockaddr_in servaddr, cliaddr;
    socklen_t cliaddr_len;
    int listenfd, connfd;
    char buf[MAXLINE];
    char str[INET_ADDRSTRLEN];
    int i, efd;
    listenfd = socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);
    bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    listen(listenfd, 20);


    struct epoll_event event;
    struct epoll_event resevent[10];
    int res, len;
    efd = epoll_create(10);
    /* ET 边沿触发，默认是水平触发*/
    event.events = EPOLLIN | EPOLLET;
    //event.events = EPOLLIN;
    printf("Accepting connections ...\n");
    cliaddr_len = sizeof(cliaddr);
    connfd = accept(listenfd, (struct sockaddr *)&cliaddr, &cliaddr_len);
    printf("received from %s at PORT %d\n",
            inet_ntop(AF_INET, &cliaddr.sin_addr, str, sizeof(str)),
            ntohs(cliaddr.sin_port));
    event.data.fd = connfd;
    epoll_ctl(efd, EPOLL_CTL_ADD, connfd, &event);


    while (1) {
        res = epoll_wait(efd, resevent, 10, -1);
        printf("res %d\n", res);
        if (resevent[0].data.fd == connfd) {
            len = read(connfd, buf, MAXLINE/2);
            write(STDOUT_FILENO, buf, len);
        }
    }
    return 0;
}
```

3.2、client.c

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#define MAXLINE 10
#define SERV_PORT 8090
int main(int argc, char *argv[])
{
    struct sockaddr_in servaddr;
    char buf[MAXLINE];
    int sockfd, i;
    char ch = 'a';
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    inet_pton(AF_INET, "192.168.152.128", &servaddr.sin_addr);
    servaddr.sin_port = htons(SERV_PORT);
    connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    while (1) {
        for (i = 0; i < MAXLINE/2; i++)
            buf[i] = ch;
        buf[i-1] = '\n';
        ch++;
        for (; i < MAXLINE; i++)
            buf[i] = ch;
        buf[i-1] = '\n';
        ch++;
        write(sockfd, buf, sizeof(buf));
        sleep(10);
    }
    Close(sockfd);
    return 0;
}
```

```
xingwenpeng@ubuntu:~/LinuxCode/epoll-13/epoll_et$ ./server
Accepting connections ...
received from 192.168.152.128 at PORT 41245
res 1
```

aaaa

4、基于网络CS非阻塞模型的epoll ET触发模式

4.1、server.c

```c
#include <stdio.h>
#include <string.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <sys/epoll.h>
#include <unistd.h>
#include <fcntl.h>
#define MAXLINE 10
#define SERV_PORT 8090


int main(void)
{
    struct sockaddr_in servaddr, cliaddr;
    socklen_t cliaddr_len;
    int listenfd, connfd;
    char buf[MAXLINE];
    char str[INET_ADDRSTRLEN];
    int i, efd, flag;
    listenfd = socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);
    bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    listen(listenfd, 20);
    struct epoll_event event;
    struct epoll_event resevent[10];
    int res, len;


    efd = epoll_create(10);
    /* ET 边沿触发，默认是水平触发*/
    event.events = EPOLLIN | EPOLLET;
```

```
    /* event.events = EPOLLIN; */
    printf("Accepting connections ...\n");
    cliaddr_len = sizeof(cliaddr);
    connfd = accept(listenfd, (struct sockaddr *)&cliaddr, &cliaddr_len);
    printf("received from %s at PORT %d\n",
            inet_ntop(AF_INET, &cliaddr.sin_addr, str, sizeof(str)),
            ntohs(cliaddr.sin_port));


    flag = fcntl(connfd, F_GETFL);
    flag |= O_NONBLOCK;
    fcntl(connfd, F_SETFL, flag);


    event.data.fd = connfd;
    epoll_ctl(efd, EPOLL_CTL_ADD, connfd, &event);
    while (1) {
        printf("epoll_wait begin\n");
        res = epoll_wait(efd, resevent, 10, -1);
        printf("epoll_wait end res %d\n", res);
        if (resevent[0].data.fd == connfd) {
            while ((len = read(connfd, buf, MAXLINE/2)) >0 )
                write(STDOUT_FILENO, buf, len);
        }
    }
    return 0;
}
```

4.2、client.c

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#define MAXLINE 10
#define SERV_PORT 8090
int main(int argc, char *argv[])
{
    struct sockaddr_in servaddr;
```

```
    char buf[MAXLINE];
    int sockfd, i;
    char ch = 'a';
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    inet_pton(AF_INET, "192.168.152.128", &servaddr.sin_addr);
    servaddr.sin_port = htons(SERV_PORT);
    connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
    while (1) {
        for (i = 0; i < MAXLINE/2; i++)
            buf[i] = ch;
        buf[i-1] = '\n';
        ch++;
        for (; i < MAXLINE; i++)
            buf[i] = ch;
        buf[i-1] = '\n';
        ch++;
        write(sockfd, buf, sizeof(buf));
        sleep(10);
    }
    Close(sockfd);
    return 0;
}
```

```
xingwenpeng@ubuntu:~/LinuxCode/epoll-13/epoll_noblock$ ./server
Accepting connections ...
received from 192.168.152.128 at PORT 41244
epoll_wait begin
epoll_wait end res 1
aaaa
bbbb
epoll_wait begin
epoll_wait end res 1
cccc
dddd
```